

Using Captain/Odhar

(For CAPT residents only)

Accessing Captain/Odhar

- If you're set up to be able to use Captain/Odhar, you should already be able to ssh in: \$ `ssh captain.nottingham.ac.uk`
 - (Otherwise speak to Phil).
- Need to be comfortable with Linux.
- Also need the VPN when off campus.
- Odhar is the newer cluster, so may be busier, in which case use Captain.
- Use it for more resource intensive/longer code.
- Also use it to back up your laptop :)

Using Captain/Odhar

- When logged in, you will need to setup your environment.
- Install any packages you need, Python (e.g. [Miniconda](#) for virtual environment management).
- To transfer files/directories, you can use [rsync](#), which will only transfer any changes.
- To move something into Captain:
 - `$rsync -avh --progress local_path captain.nottingham.ac.uk:captain_path`
- To move something out of Captain:
 - `$rsync -avh --progress captain.nottingham.ac.uk:captain_path local_path`
- `-a` gives recursion, `-v` is verbose, `-h` is human-readable format, `--progress` gives a nice progress bar.

Checking Usage

- When logged in, run the `top` command to display processes in real-time.
- It can also show system information, the types of which are configurable by you ([see manual](#)).
- You can also use `ps` to show your processes with the `-u` flag, followed by your username ([see manual](#)) .
- Make sure to check you're not using too many resources, otherwise you might get an angry email!

Running Code

- Once set up, run code as you would locally.
- You can set up VS Code to ssh into Captain/Odhar and render plots locally – see the set up document on GitHub.
- **Most important thing to remember:**
- **Set your niceness!**
- For example:
 - `$ nice -n 10 mpirun -n 2 cobaya-run test.yaml`
 - Increase the number to be nicer :)

Screens

- If you want to set your code running and then walk away, you can use `screen` ([instructions](#)).
- If you have multiple sessions, name your screens:
 - `$ screen -S screen_name`
- In your session, get a list of commands with `ctrl+a+?`.
- Set code running, then detach with `ctrl+a+d`.
- To resume: `$ screen -r`
- If you have multiple screens, use the `ls` flag to find the ID (or name) and then add it to the end of the command:
 - `$ screen -r ID`

Using The HPC

(Otherwise known as Ada)

Important links

- [SharePoint with user guides](#)
- [Request access info](#)
- [Request access form](#)
- [Linux basics for HPC](#)

Ada Tiers

- There are two tiers available, both of which are free to use:
- Exploration tier:
 - For learning and testing.
 - Simultaneous use of up to 96 CPU cores.
 - 360 GB RAM.
 - Job lengths of up to 2 days.
 - 1 TB disk space.
- Production tier:
 - For University projects – apply when ready to use the HPC for projects.
 - Simultaneous use of up to 600 CPU cores.
 - 3000 GB RAM.
 - Job lengths of up to 7 days.
 - 1 TB disk space.
 - Can apply for larger limits on CPU cores, RAM and storage.

Accessing Ada

- You will need to connect to the VPN first (both on and off campus).
- The following is for Linux (advice for Windows is on the SharePoint).
- In the terminal, ssh into one of the two login nodes (swap 01 with 02):
 - `$ ssh user@hpclogin01.ada.nottingham.ac.uk`
- Then enter your password.

Home filesystem

- Once logged in, you are in the 'home' filesystem.
- This is where you need to run your jobs from and place files.
- I recommend using `rsync` to move files between Ada and your laptop/PC.
- Move files to Ada:
 - `$rsync -avh --progress laptop_path user@hpclogin01.ada.nottingham.ac.uk:hpc_path`
- Move files from Ada:
 - `$rsync -avh --progress user@hpclogin01.ada.nottingham.ac.uk:hpc_path laptop_path`

Rsync

- Quick aside for those `rsync` flags:
 - `-a` or `--archive`: user wants recursion and to preserve almost everything.
 - `-v` or `--verbose`: more information for the user during the transfer (can give one or two of these flags for extra information).
 - `-h` or `--human-readable`: output numbers in a more human-readable format.
 - `--progress`: print information showing the progress of the transfer. This will show the percentage of the transfer completed and a timer for each file. At the end there will be a summary line.

Running jobs

- To run a job on Ada, you should follow a simple pattern:
 - Create a job script.
 - Upload the script.
 - Submit the job to the queue.
 - Check the job status.
 - Retrieve results.

Environment

- In the SLURM scripts, you will need to load modules.
- You can search for the available modules: `$ module avail`
- Just load the modules that are specific to you, for example I need to use a Python file with openmpi, so I use:
 - `module load openmpi-uoneasy/4.16-GCC-13.2.0`
- I would recommend creating a new virtual python environment and if any modules are required for your code, load them before you create the environment.

SLURM scripts

- You must submit jobs from login nodes to compute nodes using these scripts.
- These will tell the system what hardware requirements you have.
- Run a job on Ada using the '`sbatch`' command followed by the SLURM script name.

Example scripts:

```
#!/bin/bash
#SBATCH --partition=shortq
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --mem=2g
#SBATCH --time=00:10:00
echo "Hello from $(hostname)!"
echo "Current date and time: $(date)"
sleep 30
echo "Job completed!"
```

```
#!/bin/bash
# Example submission script for 'hello world' program

# This specifies type of node job will use #SBATCH --nodes=1
#SBATCH --partition=defq

# This specifies job uses 1 node
#SBATCH --ntasks-per-node=1

# This specifies maximum memory use will be 2 gigabytes
#SBATCH --mem=2g

# This specifies job will last no longer than 1 hour
#SBATCH --time=1:00:00

#below use Linux commands, which will run on compute node. This needs to be specific to your
#application

echo "Running on `hostname`"
cd ${SLURM_SUBMIT_DIR}
./hello

sleep 40

echo "Finished job now"
```


SLURM scripts

- Any options for the `sbatch` command are preceded by `#SBATCH`.
- Commands:
 - `--partition` = specifies the type of node being used:
 - `defq`: standard jobs, up to 1 week
 - `shortq`: short jobs, up to 12 hours.
 - `hmemq`: high memory nodes
 - `ampereq`: GPUs
 - Etc. ([see here](#))
 - `--nodes` = number of computers to request
 - `--ntasks-per-node` = number of CPU cores
 - `--mem` = amount of memory needed
 - `--time` = estimated maximum time
 - `--cpus-per-task` = number of cores for each task (threading)
 - `--mail-type` = when to be emailed about the job (e.g. `BEGIN`, `END`, `FAIL`)
 - `--mail-user` = email address
 - `--job-name` = custom job name
 - `--output` = custom output file name (for standard output and error)

Submitting Jobs

- Submit a job with: `$ sbatch myjob.sh`
- The commands in the script can also be used in the CLI :
 - `$ sbatch --nodes=1 --ntasks-per-node=1 myjob.sh`
 - This overrides the `#SBATCH` option.
- Try to submit resources requirements accurately, so that the scheduler can work efficiently.

Monitoring jobs

- You can display a summary of the jobs on the system with:

- `$ squeue`

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)	
	2053	ampereq	l13c_0	cczcb	PD	0:00	1	(Dependency)
	2052	ampereq	l13c_0	cczcb	R	11:40	1	ampere001
	2056	defq	myjob-la	cczcb	PD	0:00	57	(Resources)
	2057	defq	myjob-lo	uizlp	R	4:04	1	comp007
	2059	shortq	f2m40cRi	cczcb	R	0:05	2	comp[005-006]

- Flags include: `-u` for specific users and `-l` for more information:

- `$ squeue -l -u cczcb`

Wed Nov 08 14:21:34 2023									
JOBID	PARTITION	NAME	USER	STATE	TIME	TIME_LIMI	NODES	NODELIST(REASON)	
2053	ampereq	l13c_0	cczcb	PENDING	0:00	15:00:00	1	(Dependency)	
2052	ampereq	l13c_0	cczcb	RUNNING	13:21	15:00:00	1	ampere001	
2056	defq	myjob-la	cczcb	PENDING	0:00	1:00:00	57	(Resources)	
2059	shortq	f2m40cRi	cczcb	RUNNING	1:46	12:00:00	2	comp[005-006]	

- Use: `$ man squeue` for information on the options available.
- Once a job finishes, it disappears from `squeue`.

Monitoring jobs

- To remove a queued or running job from the job queue, use the `scancel` command followed by the job ID:
 - `$ scancel 28463`
- `scontrol` gives detailed information on a queued or running job:
 - `$ scontrol show job 216683`

```
JobId=216683 JobName=myjob.sh
  UserId=cczcb(6206707) GroupId=domain users(6200513) MCS_label=N/A
  Priority=100033227 Nice=0 Account=ui-uonhpcaccess QOS=hpc
  JobState=RUNNING Reason=None Dependency=(null)
  Requeue=1 Restarts=0 BatchFlag=1 Reboot=0 ExitCode=0:0
  RunTime=00:00:12 TimeLimit=01:00:00 TimeMin=N/A
  SubmitTime=2018-10-03T14:54:57 EligibleTime=2018-10-03T14:54:57
  StartTime=2018-10-03T14:54:58 EndTime=2018-10-03T15:54:58 Deadline=N/A
  PreemptTime=None SuspendTime=None SecsPreSuspend=0
  Partition=defq AllocNode:Sid=login003:146573 ReqNodeList=(null)
  ExcNodeList=(null) NodeList=compute011 BatchHost=compute011
  NumNodes=1 NumCPUs=1 NumTasks=1 CPUs/Task=1 ReqB:S:C:T=0:0:*:*
  TRES=cpu=1,mem=2G,node=1 Socks/Node=* NtasksPerN:B:S:C=1:0:*:* CoreSpec=*
  MinCPUsNode=1 MinMemoryNode=2G MinTmpDiskNode=0 Features=(null)
  DelayBoot=00:00:00 Gres=(null) Reservation=(null) OverSubscribe=OK Contiguous=0
  Licenses=(null) Network=(null) Command=/gpfs01/home/cczcb/testrun/serial/myjob.sh
  WorkDir=/gpfs01/home/cczcb/testrun/serial StdErr=/gpfs01/home/cczcb/testrun/serial
  slurm-216683.out StdIn=/dev/null StdOut=/gpfs01/home/cczcb/testrun/serial
  slurm-216683.out
  Power=
```

Monitoring jobs

- `sacct` is the best command for summarising information on running and completed jobs.
- By default, it shows jobs run in the same day, but this can be changed with the `-S` and `-E` flags.

JobID	JobName	Partition	Account	AllocCPUS	State	ExitCode
19664494	run.slurm	defq	ui-uonhpc+	35	COMPLETED	0:0
19664494.ba+	batch		ui-uonhpc+	35	COMPLETED	0:0
19664494.ex+	extern		ui-uonhpc+	35	COMPLETED	0:0
19664494.0	hydra_pmi+		ui-uonhpc+	35	COMPLETED	0:0

- You can choose specific jobs with `-j` and specific information on the job selected, e.g.:
 - `$ sacct -o JobID,JobName,Start,End,Partition,Account,NNodes,ReqNodes,ReqCPUS,NTasks,NCPUS,CPUTime,Elapsed,TimeLimit -j 1261`

Managing files

- Just make sure to clean up files you don't need and what you need to transfer to your laptop.
- The scheduler will produce a file containing standard output and error.
- Typically, this is "`slurm-job_id.out`", where `job_id` can be found with `sacct` as before.
 - You can change the name of the file in your submission script.

Parallel code

- You should test how the code 'scales' in performance with multiple cores and nodes.
- Avoid wasting resources.
- Consider how much your code will speed up when increasing the number of cores and whether it is worth it.